Security

Part 3 of 5: passwords, tokens, roles, and the tenancy boundary

FILES: SECURITY/ · SERVICES/USERSERVICE.CS · SERVICES/ACCESSGUARD.CS · ENDPOINTS/*FILTER.CS

1. Passwords

Passwords are never stored. What is stored is a **hash** — a one-way transformation. Given the hash you cannot recover the password; you can only hash a guess and compare.

```
private readonly PasswordHasher<User> _passwordHasher = new();
```

`PasswordHasher<User>` ships with ASP.NET Core and uses PBKDF2 with a per-password random salt and a high iteration count. Two properties matter:

- **Salt.** Two users with the same password get different hashes, so cracking one does not crack the other, and precomputed rainbow tables are useless.
- **Slowness.** Deliberate. A hash that takes 100 ms is invisible at login and ruinous for an attacker trying millions of guesses.

The verify step has a third state beyond pass/fail:

```
if (result == PasswordVerificationResult.SuccessRehashNeeded)
{
    user.PasswordHash = _passwordHasher.HashPassword(user, dto.Password);
    ...
}
```

The password was right, but hashed with older parameters. Since the plaintext is in hand for this one instant, it is rehashed with current settings. This is how a system upgrades its stored hashes over time without asking anyone to reset anything.

The login flow, and why the order matters

```
var user = await userRepository.GetByEmailAsync(dto.Email, ct);

if (user is null)
    throw new UnauthorizedException("Invalid email or password.");

var result = _passwordHasher.VerifyHashedPassword(user, user.PasswordHash, dto.Password);
if (result == PasswordVerificationResult.Failed)
    throw new UnauthorizedException("Invalid email or password.");

if (user.DeactivatedAt is not null)
    throw new UnauthorizedException("This account has been deactivated.");
```

Two deliberate choices here, both worth stating in a defence:

1. **Identical message** for unknown email and wrong password. Distinguishing them turns the login form into a tool for discovering which addresses have accounts.
2. **The deactivation check runs last**, after the password is verified. Check it first and you leak account status to someone who cannot prove they own the account.

Password rules

One rule set in `Validation/PasswordRules.cs` is shared by every entry point — user-chosen and admin-created alike:

```
public static IRuleBuilderOptions<T, string> Password<T>(this IRuleBuilder<T, string> rule) =>
    rule.NotEmpty().Length(8, 128)
        .Matches(@"[a-z]").Matches(@"[A-Z]").Matches(@"\d");
```

These had drifted apart: users changing their own password needed 8+ characters with mixed case and a digit, while an admin creating a student could set a one-character password. One shared rule set removes the possibility of drift.

Admin password reset

`AdminResetPasswordAsync` generates a random password, stores its hash, sets `MustChangePassword = true`, and returns the plaintext **once** for the admin to hand over. It is never stored in readable form. The recipient cannot use the account for anything else until they replace it — enforced in section 5.

2. What a JWT actually is

A JSON Web Token is three base64 segments separated by dots: `header.payload.signature`.

> The payload is **encoded, not encrypted**. Anyone can paste a token into jwt.io and read every claim. Never put a secret in one. What a JWT gives you is *integrity* — proof the contents were issued by you and have not been altered — not confidentiality.

The signature is an HMAC-SHA256 of the header and payload using a server-side secret key. Change one byte of the payload and the signature no longer matches. Since only the server has the key, only the server can mint a valid token.

The claims EduCore issues

```
var claims = new List<Claim>
{
    new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
    new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
    new(JwtRegisteredClaimNames.Email, user.Email),
    new(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new(ClaimTypes.Role, user.Role.ToString()),
    new(ClaimNames.MustChangePassword, user.MustChangePassword ? "true" : "false")
};

if (schoolId  is not null) claims.Add(new Claim(ClaimNames.SchoolId,  schoolId.Value.ToString()));
if (teacherId is not null) claims.Add(new Claim(ClaimNames.TeacherId, teacherId.Value.ToString()));
if (studentId is not null) claims.Add(new Claim(ClaimNames.StudentId, studentId.Value.ToString()));
```

CLAIM	PURPOSE
`sub` / `NameIdentifier`	Which user. Read by `ICurrentUser.UserId` and by the audit interceptor.
`jti`	Unique id for this token, so an individual one can be traced in logs.
`role`	Drives every `.RequireAuthorization(...)` policy.
`school_id`	**The tenancy boundary.** Read by every query filter and by the write interceptor.
`teacher_id` / `student_id`	Lets a service ask “is this your own record?” without a database round-trip.
`must_change_password`	Restricts a token issued after an admin reset.

Every one of these is **server-minted**. Nothing anywhere reads a school, teacher or student id from a request body, header or query string. That is what makes the boundary trustworthy: a caller can choose what to *ask* for, never who they *are*.

Validation on the way back in

```
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = true,           ValidIssuer   = ...["Jwt:Issuer"],
    ValidateAudience = true,         ValidAudience = ...["Jwt:Audience"],
    ValidateIssuerSigningKey = true, IssuerSigningKey = new SymmetricSecurityKey(...),
    ValidAlgorithms = [SecurityAlgorithms.HmacSha256],
    ValidateLifetime = true,         ClockSkew = TimeSpan.FromMinutes(1)
};
```

Line by line: the token must have been issued by us, for this API, signed with our key, using the algorithm we expect, and not expired. `ClockSkew` tolerates a minute of clock drift between machines. Pinning `ValidAlgorithms` is defence in depth against algorithm-substitution attacks.

And the key itself is checked at startup:

```
if (string.IsNullOrWhiteSpace(jwtKey) || Encoding.UTF8.GetByteCount(jwtKey) < 32)
    throw new InvalidOperationException("Jwt:Key must be at least 32 bytes. ...");
```

HS256 needs 256 bits of key. Without this the app would happily start and sign real tokens with the literal string `CHANGE_ME`. Failing loudly at boot is far better than failing silently in production.

3. Reading the caller: `ICurrentUser`

One small class turns claims into typed properties, so no service ever parses a claim itself:

```
public Guid? SchoolId => ReadGuid(ClaimNames.SchoolId);
public bool IsSuperAdmin => Role == UserRole.SuperAdmin;

private Guid? ReadGuid(string claimType) =>
    Guid.TryParse(Principal?.FindFirst(claimType)?.Value, out var value) ? value : null;
```

Note the return types. `SchoolId` is `Guid?` because a SuperAdmin genuinely has no school; the nullability is the domain, not laziness. `UserId` is non-nullable and throws if absent, because any code reaching it is already behind authentication.

`AppDbContext` depends on this interface too — that is how the query filters know which school to filter by.

4. Authorization, in three layers

The same three-question framing from Part 1, now in detail.

Layer 1 — role policies

`Security/Policies.cs`

```
options.AddPolicy(AdminOrTeacher, p =>
    p.RequireRole(Role(UserRole.Admin), Role(UserRole.Teacher)));
```

Used as `.RequireAuthorization(Policies.AdminOrTeacher)`. This runs *before* your endpoint code, so a student's token never reaches the handler. Naming the policies in one file rather than repeating role strings means a typo is a compile error, not a silently open endpoint.

One subtlety worth knowing: `AdminOrTeacher` and `AnySchoolUser` both *exclude* SuperAdmin. That is intentional — a platform operator is not a teacher — but it does mean a SuperAdmin cannot log lessons or give marks.

Layer 2 — the school query filter

Covered in Part 2. Automatic, unforgettable, and fails towards seeing nothing.

Layer 3 — `AccessGuard`

Roles and tenancy are both broad. Neither can stop one teacher reading another teacher's class inside the same school. `AccessGuard` answers exactly that, with three methods:

METHOD	ADMIN	TEACHER	STUDENT
`EnsureCanViewClassAsync`	any class in school	classes they teach or are homeroom of	their own class
`EnsureCanTeachClassAsync`	any class in school	only classes assigned to them	never
`EnsureCanViewStudentAsync`	any student in school	students in a class they teach	only themselves

```
public async Task EnsureCanTeachClassAsync(Guid classId, CancellationToken ct)
{
    _ = await classes.GetByIdAsync(classId, ct) ?? throw new NotFoundException("Class", classId);

    if (currentUser.IsAdmin || currentUser.IsSuperAdmin)
        return;

    if (currentUser.IsTeacher
        && await assignments.TeachesClassAsync(currentUser.RequireTeacherId(), classId, ct))
        return;

    throw new ForbiddenException("You are not assigned to teach this class.");
}
```

> **Why this is one class and not an `if` in each service.** These same three questions arise in six services. When the answer is written out six times, five are right and one is missing. That is not hypothetical: `EnsureCanTeachClassAsync` was written, implemented, and never called, which left `POST /api/diary-entries` with no class check at all. Any teacher could open a register for any class in the school and then mark attendance and write behaviour notes about children they had never taught. One place to look is one place to be wrong.

A subtlety in `EnsureCanViewStudentAsync`: a teacher's reach is derived from the student's *current* enrollment. When a student transfers out, that teacher loses access — including to past marks. Defensible, but you should know it is a choice.

5. The must-change-password gate

An admin resets someone's password. That person now holds a valid token for an account whose password they did not choose. They should be able to do exactly one thing.

```
public class PasswordChangeEndpointFilter : IEndpointFilter
{
    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context, EndpointFilterDelegate next)
    {
        if (context.HttpContext.User.FindFirst(ClaimNames.MustChangePassword)?.Value == "true")
            throw new ForbiddenException("You must change your password before using the
application.");

        return await next(context);
    }
}
```

Applied once to the whole `/api` group in `Program.cs`. The change-password endpoint lives outside that group, so it stays reachable. This makes the flag an *enforced* rule rather than a hint the front end is trusted to honour.

6. Validation

`ValidationEndpointFilter` looks up a registered FluentValidation validator for each request body and runs it:

```
foreach (var argument in context.Arguments)
{
    if (argument is null || argument.GetType().Namespace?.StartsWith("EduCore.Dtos") != true)
        continue;

    if (services.GetService(typeof(IValidator<>).MakeGenericType(argument.GetType())) is not IValidator
validator)
        continue;

    var result = await validator.ValidateAsync(new ValidationContext<object>(argument));
    errors.AddRange(...);
}

if (errors.Count > 0) throw new RequestValidationException(errors);
```

Registered once on the group, so no endpoint can forget to validate. The namespace check skips injected services and the cancellation token.

The division of labour is worth stating clearly: **validators check shape, services check meaning.** "Value is between 1 and 5" is a validator. "This grading period belongs to this class's academic year" needs the database, so it is a service rule that throws `BusinessException`.

7. Every cross-tenant escape hatch, listed

`.IgnoreQueryFilters()` deliberately disables tenant isolation. There are nine calls. Being able to enumerate and justify them is the strongest possible answer to "how do you know schools are isolated?"

WHERE	COUNT	WHY IT IS SAFE
`AdminRepository`	4	Admins are managed by the SuperAdmin, who has no school. All four are reachable only from SuperAdmin-gated endpoints.

`StudentRepository.GetByUserIdAsync`
`TeacherRepository.GetByUserIdAsync`
`AdminRepository.GetByUserIdAsync`
	3	Run during login, *before* a school claim exists. The filter would hide the very row that says which school the person belongs to.

`StudentRepository.PersonalNumberExistsAsync`	1	A national ID is globally unique by law, so the check must span schools. Returns a bool only, never a row.

The remaining one is the `Admin` lookup used by both the login path and the SuperAdmin endpoints. Every call is gated by role at the endpoint, and none returns another school's data to a school user.

8. Known gaps — state these before you are asked

Volunteering known limitations reads as competence. Hiding them and being caught does not.

Deactivation does not revoke live tokens

All authorization state lives in the token. Deactivating a user stops future logins but does not invalidate a token already issued, so a disabled account keeps working until expiry (default 60 minutes). **Fix:** a `SecurityStamp` column bumped on deactivate and on every password change, emitted as a claim and checked in `JwtBearerEvents.OnTokenValidated`. Cost: one indexed read per request.

No rate limiting on login

`POST /api/auth/login` is anonymous and unthrottled, so credentials can be sprayed at it. Deliberately slow password hashing makes this a CPU-exhaustion vector too. **Fix:** ASP.NET Core rate limiting keyed on IP and submitted email, plus per-account lockout after N failures.

Teachers can read national IDs

`StudentListDto` omits `PersonalNumber` to avoid bulk exposure, but `StudentDetailDto` includes it and any teacher may fetch students they teach, one at a time. Whether that is correct is a policy question about how the school actually operates, not a bug.

Concurrency conflicts surface as 500

`DbUpdateConcurrencyException` is not mapped in `GlobalExceptionHandler`. It should return 409 Conflict.

The API has never been run against a live database

It compiles cleanly, the DI container validates, and AutoMapper's configuration is verified at startup — but no endpoint has been exercised end to end, because no PostgreSQL instance was available. Say this plainly if asked what is untested.